Adaptive Hybrid SAT-HCP

Từ nhập môn SAT đến thiết kế một phương pháp tích hợp

Tài liệu này giải thích bài toán chu trình Hamilton, SAT, CEGAR, cut-set và user propagator bằng trực giác trước, công thức sau. Phần cuối trình bày một phương pháp duy nhất có khả năng học và tận dụng các kỹ thuật tốt nhất từ những baseline hiện có.

Dự án HCP-CEGAR

Phiên bản thiết kế, tháng 8 năm 2026

Cách đọc tài liệu

Tài liệu được viết cho cả người chưa từng học SAT. Người đọc không cần biết logic mệnh đề, solver hay CEGAR trước đó.

- Nếu mới bắt đầu, hãy đọc tuần tự từ Chương 1 đến Chương 4.
- Nếu đã biết SAT, có thể bắt đầu từ Chương 4 về CEGAR và cut-set.
- Nếu quan tâm trực tiếp đến phương pháp mới, đọc Chương 6 đến Chương 10.
- Các hộp màu xanh giải thích trực giác; hộp cam nêu rủi ro; hộp viền đậm chốt kết luận.

Trạng thái khoa học của đề xuất

Các con số trong Chương 5 đến từ dữ liệu đã có. Kiến trúc Adaptive Hybrid SAT-HCP trong các chương sau là một *đề xuất nghiên cứu*: các module thành phần đã tồn tại hoặc đã có bằng chứng riêng, nhưng lợi ích của toàn bộ hệ tích hợp vẫn phải được kiểm nghiệm bằng một benchmark công bằng.

Mục lục

Cách đọc tài liệu i

1. Bài toán chu trình Hamilton là gì? 1
1.1. Đồ thị: thành phố và con đường . . . 1
1.2. Tại sao bài toán khó? . . . 1

2. SAT cho người mới bắt đầu 2
2.1. SAT là bài toán những công tắc đúng hoặc sai . . . 2
2.2. Literal, clause và công thức CNF . . . 2
2.3. SAT solver làm việc như thế nào? . . . 2
2.4. SAT, UNSAT và UNKNOWN . . . 3

3. Biến HCP thành SAT 4
3.1. Ràng buộc bậc: mỗi đỉnh chọn đúng hai cạnh . . . 4
3.2. Vấn đề cycle cover . . . 4
3.3. Tại sao không thêm mọi ràng buộc ngay từ đầu? . . . 4

4. CEGAR và cut-set bằng trực giác 5
4.1. CEGAR là vòng lặp "đoán, kiểm tra, sửa" . . . 5
4.2. Blocking subcycle và cut-set khác nhau thế nào? . . . 5
4.3. 2-opt và 3-opt: sửa lời giải thay vì bỏ đi . . . 5

5. Ta học được gì từ các baseline? 7
5.1. Phân biệt cấu hình đơn, portfolio và VBS . . . 7
5.2. Bức tranh dữ liệu hiện có . . . 7
5.3. Bài học cụ thể từ từng baseline . . . 8

6. Phương pháp đề xuất: Adaptive Hybrid SAT-HCP 10
6.1. Một phương pháp, không phải một túi solver . . . 10
6.2. Năm nguyên tắc thiết kế . . . 10
6.3. Kiến trúc tổng thể . . . 10
6.4. Phase A: CEGAR có inprocessing . . . 11
6.5. Treatment đã triển khai: population-guided CEGAR . . . 12
6.6. Phase B: SAT-guided repair . . . 13
6.7. Phase C: prevent-subcycle user propagation . . . 13
6.8. Ứng viên encoding rebuild: CRE–CEGAR . . . 13
6.9. Vì sao cần rebuild giữa Phase A và Phase C? . . . 15

7. Hybrid Controller quyết định như thế nào? 16
7.1. Telemetry tối thiểu . . . 16
7.2. Controller đang chạy: luật cố định, không học máy . . . 16
7.3. Phân bổ ngân sách . . . 17

8. Các hướng brainstorming có tiềm năng 18
8.1. Encoding lai theo từng đỉnh . . . 18
8.2. Bốn đề xuất từ nhánh SAT encoding . . . 18
8.2.1. Preprocessing khoảng cách an toàn cho CRE . . . 18

8.2.2. ExactlyOne: dùng policy thay vì khóa vào PBLib 19
8.3. GraphCutPool độc lập encoding 19
8.4. Phase hints từ cycle cover tốt nhất 19
8.5. Tổng quát hóa cut theo xung đột 19
8.6. Hướng tương lai chưa triển khai: học controller bằng dữ liệu 19

9. Thiết kế thực nghiệm công bằng 21
9.1. Câu hỏi nghiên cứu 21
9.2. Ba mức kết quả phải báo cáo riêng 21
9.3. Tập đánh giá trọng tâm 21
9.4. Chỉ số đánh giá 22
9.5. Ablation bắt buộc 22

10. Lộ trình triển khai 24
10.1. Giai đoạn 0: hoàn thiện bằng chứng 24
10.2. Giai đoạn 1: GraphCutPool và telemetry 24
10.3. Giai đoạn 1a: ablate population-guided CEGAR 24
10.4. Giai đoạn 2: kiểm chứng và ablate CRE–CEGAR 24
10.5. Giai đoạn 3: làm cứng và ablate Phase B 25
10.6. Giai đoạn 4: driver A–C và tích hợp HCup 25
10.7. Giai đoạn 5: mở rộng controller luật; selector học là tùy chọn tương lai 25

11. Pseudocode dễ đọc 26

12. Câu hỏi thường gặp 27

A. Bảng thuật ngữ 28

B. Tái tạo báo cáo coverage và benchmark 29

C. Tài liệu tham khảo 31

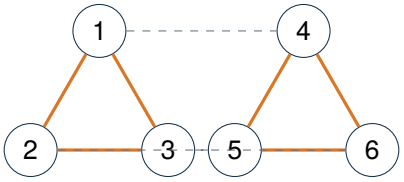
1. Bài toán chu trình Hamilton là gì?

1.1. Đồ thị: thành phố và con đường

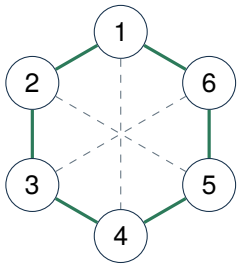
Một *đồ thị* gồm các đỉnh và các cạnh. Có thể hình dung đỉnh là thành phố, cạnh là con đường nối hai thành phố. Trong tài liệu này, ta xét đồ thị vô hướng: nếu đi được từ u đến v thì cũng đi được từ v đến u.

Một *chu trình Hamilton* là một chuyến đi:

1. đi qua mọi đỉnh đúng một lần;
2. sau cùng quay về đỉnh xuất phát;
3. chỉ sử dụng những cạnh có trong đồ thị.



(a) Mỗi đỉnh có bậc 2 trong lời giải, nhưng có hai chu trình rời nhau.



(b) Một chu trình duy nhất đi qua cả sáu đỉnh: đây là chu trình Hamilton.

Hình 1.1.: Khác biệt giữa một cycle cover và một chu trình Hamilton.

Trực giác

Điều khó không phải chỉ là chọn hai cạnh tại mỗi đỉnh. Điều khó là buộc tất cả các cạnh được chọn phải nối thành *một* vòng duy nhất. Nhiều vòng nhỏ như hình bên trái là một lời giải gần đúng nhưng chưa hợp lệ.

1.2. Tại sao bài toán khó?

Với một đồ thị nhỏ, ta có thể thử từng đường đi. Khi số đỉnh tăng, số khả năng tăng quá nhanh. hcp là bài toán NP-complete: chưa có thuật toán tổng quát nào được biết có thể giải nhanh mọi trường hợp.

Điều này không có nghĩa mọi instance đều khó. Nhiều đồ thị lớn được giải trong tích tắc, trong khi một số đồ thị nhỏ hơn có cấu trúc đối xứng hoặc nhiều lựa chọn gần giống nhau lại làm solver mất hàng chục phút.

2. SAT cho người mới bắt đầu

2.1. SAT là bài toán những công tắc đúng hoặc sai

Trong SAT, mỗi biến chỉ có hai giá trị:

$$x = \text{Đúng} \qquad \text{hoặc} \qquad x = \text{Sai}.$$

Với HCP, ta tạo một biến x_{uv} cho mỗi cạnh (u, v):

$$x_{uv} = \text{Đúng} \iff \text{cạnh } (u, v) \text{ được chọn vào chu trình.}$$

SAT solver có nhiệm vụ bật hoặc tắt hàng nghìn, thậm chí hàng triệu công tắc sao cho mọi quy tắc đều được thỏa mãn.

2.2. Literal, clause và công thức CNF

Một *literal* là một biến hoặc phủ định của biến:

$$x \quad \text{hoặc} \quad \neg x.$$

Một *clause* là phép "hoặc". Clause

$$(x_{12} \vee x_{13})$$

nói rằng phải chọn ít nhất một trong hai cạnh $(1, 2)$ và $(1, 3)$.

Một công thức CNF là phép "và" của nhiều clause:

$$(x_{12} \vee x_{13}) \wedge (\neg x_{12} \vee x_{14}) \wedge (\neg x_{13} \vee \neg x_{14}).$$

Ví dụ ba công tắc

Giả sử x_{12} được bật. Clause thứ hai buộc x_{14} cũng bật. Khi đó clause thứ ba buộc x_{13} tắt. SAT solver thực hiện hàng loạt suy luận kiểu này, nhưng ở quy mô rất lớn và có cơ chế quay lui khi đi vào ngõ cụt.

2.3. SAT solver làm việc như thế nào?

Một solver hiện đại như CaDiCaL có thể được hình dung bằng bốn thao tác:

1. **Quyết định:** thử gán một biến là đúng hoặc sai.
2. **Lan truyền:** từ các clause hiện có, suy ra những biến bắt buộc.

3. **Phát hiện xung đột:** nhận ra rằng một nhánh lựa chọn không thể thỏa mãn.
4. **Học clause và quay lui:** ghi nhớ nguyên nhân xung đột để không lặp lại sai lầm tương tự.

SAT solver không chỉ "thử hết"

Điểm mạnh của SAT solver là học từ thất bại. Mỗi xung đột có thể tạo ra một quy tắc mới, loại bỏ cả một vùng lớn của không gian tìm kiếm thay vì chỉ loại một nghiệm.

2.4. SAT, UNSAT và UNKNOWN

Kết quả	Ý nghĩa
SAT	Tìm được một phép gán thỏa mọi clause. Với HCP, vẫn cần kiểm tra tour đầu ra.
UNSAT	Chứng minh không có phép gán nào thỏa công thức.
UNKNOWN/TIMEOUT	Chưa giải xong trong giới hạn tài nguyên; không được hiểu là UNSAT.

3. Biến HCP thành SAT

3.1. Ràng buộc bậc: mỗi đỉnh chọn đúng hai cạnh

Trong một chu trình Hamilton vô hướng, mỗi đỉnh có đúng hai cạnh được chọn: một cạnh để đi vào và một cạnh để đi ra. Với đỉnh v, ta cần:

$$\sum_{e \in \delta(v)} x_e = 2,$$

trong đó $\delta(v)$ là tập cạnh kề với v.

Máy SAT không hiểu trực tiếp phép cộng này, nên ta phải dịch nó thành các clause. Có nhiều cách dịch:

- **Pairwise/Binomial:** dễ hiểu, tốt khi số cạnh kề ít, nhưng sinh nhiều clause khi bậc lớn.
- **Sinz sequential counter:** thêm các biến phụ theo kiểu bộ đếm tuần tự; thường gọn và lan truyền tốt.
- **Commander:** chia biến thành nhóm rồi dùng biến chỉ huy; hữu ích cho tập biến lớn.
- **Adder/CRT:** mô phỏng phép cộng hoặc thứ tự bằng mạch logic; có ưu điểm ở một số cấu trúc nhưng không phải lúc nào cũng tốt nhất.

3.2. Vấn đề cycle cover

Chỉ ràng buộc "mỗi đỉnh đúng hai cạnh" chưa đủ. SAT solver có thể trả về nhiều chu trình nhỏ rời nhau như Hình 1.1. Kết quả đó gọi là một *cycle cover*.

Ta cần thêm ràng buộc kết nối toàn cục: không được tồn tại một tập đỉnh S bị cô lập khỏi phần còn lại của chu trình.

3.3. Tại sao không thêm mọi ràng buộc ngay từ đầu?

Với mỗi tập con $S \subset V$, về lý thuyết ta có thể viết một ràng buộc ngăn S trở thành chu trình riêng. Nhưng số tập con là cấp số mũ. Việc sinh toàn bộ trước khi giải có thể tạo công thức quá lớn.

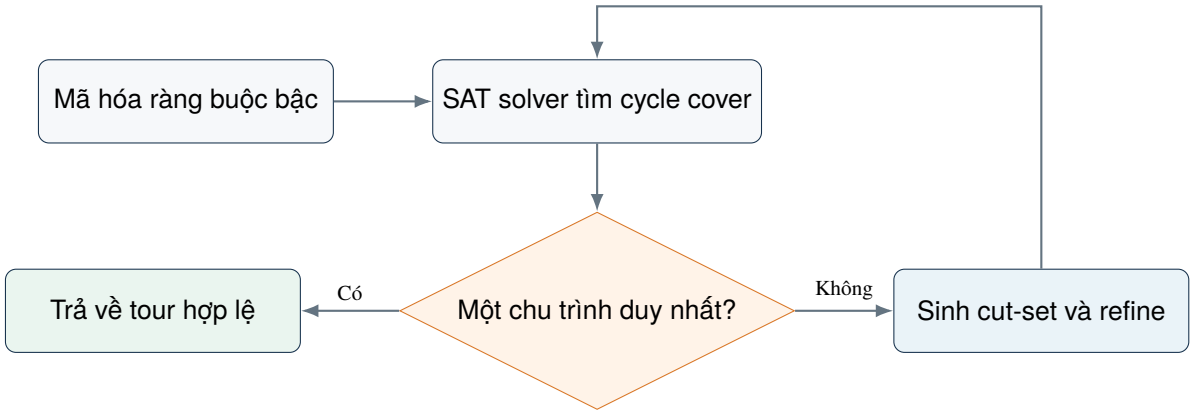
Kết luận chính

Ý tưởng cốt lõi của CEGAR là: ban đầu chỉ đưa vào SAT phần dễ và nhỏ; khi solver cho một lời giải sai, ta mới bổ sung đúng ràng buộc cần thiết để loại kiểu sai đó.

4. CEGAR và cut-set bằng trực giác

4.1. CEGAR là vòng lặp "đoán, kiểm tra, sửa"

CEGAR là viết tắt của *Counterexample-Guided Abstraction Refinement*. Với HCP, quy trình có thể hiểu như sau:



Hình 4.1.: Vòng lặp CEGAR cho HCP.

4.2. Blocking subcycle và cut-set khác nhau thế nào?

Giả sử solver tạo ra một chu trình con trên tập đỉnh S.

Blocking subcycle nói: "Không được chọn lại chính xác các cạnh tạo ra chu trình con này." Cách này có thể chỉ loại một cấu hình rất cụ thể.

Cut-set nhìn rộng hơn. Gọi $\delta(S)$ là tập cạnh nối một đỉnh trong S với một đỉnh ngoài S. Một chu trình Hamilton phải đi ra và quay lại S, nên cần ít nhất hai cạnh qua biên:

$$\sum_{e \in \delta(S)} x_e \geq 2.$$

Ví dụ hòn đảo

Hãy hình dung S là một hòn đảo. Một tour đi qua toàn bộ bản đồ phải có đường vào và đường ra khỏi đảo. Cut-set không chỉ cấm một con đường nội bộ cụ thể; nó buộc lời giải phải kết nối đảo với bên ngoài.

4.3. 2-opt và 3-opt: sửa lời giải thay vì bỏ đi

Khi SAT trả về nhiều chu trình, ta không nhất thiết loại ngay lời giải đó. Nếu hai chu trình có các cạnh chéo thích hợp, 2-opt có thể tháo hai cạnh và nối lại để hợp nhất chúng. Restricted 3-opt thử một phép nối phức tạp hơn với ba cạnh.

Nếu sửa thành công, ta có lời giải mà không cần gọi SAT thêm. Nếu chỉ sửa được một phần, cycle cover đã cải thiện có thể cung cấp thông tin cho bước refine.

Điểm cần cẩn thận

Patcher là heuristic: nó có thể giúp rất nhiều trên một họ graph nhưng tốn thời gian hoặc dẫn tìm kiếm về cùng một attractor trên họ khác. Vì vậy phương pháp mới không nên bật mọi patcher vô điều kiện.

5. Ta học được gì từ các baseline?

5.1. Phân biệt cấu hình đơn, portfolio và VBS

Ba khái niệm thường bị nhầm lẫn:

Khái niệm	Cách hoạt động	Có thể chạy như một solver thật?
Cấu hình đơn	Một bộ tham số cố định cho mọi instance.	Có.
Portfolio tĩnh	Thử nhiều cấu hình theo thứ tự cố định, dùng chung một ngân sách được quy định trước.	Có, nếu ngân sách tổng được tính công bằng.
VBS	Sau khi biết mọi kết quả, chọn hồi tố cấu hình tốt nhất cho từng instance.	Không. Đây là oracle và chỉ là cận trên.

5.2. Bức tranh dữ liệu hiện có

Dữ liệu cục bộ và các tổng do hai bài báo công bố cho thấy:

Kết quả	Số instance	Cách diễn giải
Bài Ohashi et al. 2025, cấu hình `-asym`	937/1001	Kết quả cấu hình đơn do bài báo công bố, với cap 1800 giây.
Baseline cục bộ chính thức	926/1001	Lần chạy trên mã nguồn đã chỉnh sửa với `--three-opt 1`; đây không phải kết quả gốc của bài SAT 2025.
SAT-CEGAR tốt nhất, một cấu hình	957/1001	Mốc thực tế mạnh nhất trong các cột CEGAR đã ghi nhận.
Union của 16 biến thể SAT-CEGAR	963/1001	VBS trong họ SAT-CEGAR; chưa phải một solver đơn.
Union của 21 phương pháp có log đầy đủ	968/1001	Portfolio giải thích union này lần lượt nhận thêm 8 graph từ ASP, 2 từ Sinz và 1 từ Picat. Các số này phụ thuộc thứ tự portfolio.
Core chưa được 21 phương pháp giải	33/1001	Chưa thể kết luận HCup cũng timeout vì chưa có log HCup per-instance cục bộ.
HCup outside-mergecutset	937/1001	Tổng do bài HCup báo cáo; chưa có log per-instance cục bộ để tính overlap.
HCup hfp-preventsubcycle	917/1001	Cấu hình user propagator đơn mạnh nhất trong báo cáo HCup.
HCup VBS-propagator	961/1001	Oracle trên nhiều cấu hình, không phải một solver đơn.

Bảng 5.1.: Các mốc coverage cần phân biệt khi thiết kế phương pháp mới.

Log regression 48 giờ ở cap 300 giây bị ngắt giữa task 286: có 285 task hoàn thành, một task đã bắt đầu nhưng chưa có kết quả và 194 task chưa chạy. Trong phần hoàn thành có 26 run SAT thuộc sáu graph, nhưng tất cả các graph đó đã được phương pháp cũ giải. Log này chưa bổ sung coverage và chưa giải instance core-33 nào. Task chưa chạy phải ghi là `MISSING`, không phải `TIMEOUT`.

Thông điệp từ các con số

Dữ liệu cục bộ cho thấy encoding và một số baseline có lời giải bổ sung cho nhau. Riêng tính bổ sung giữa CEGAR và HCup chưa được đo theo từng instance. Vì vậy, kết hợp CEGAR, model repair và user propagation là giả thuyết trung tâm cần kiểm nghiệm, chưa phải kết quả đã chứng minh.

5.3. Bài học cụ thể từ từng baseline

Nguồn	Kỹ thuật đáng học	Cách đưa vào phương pháp chung
CEGAR cũ	Incremental SAT và subtour elimination.	Là backbone bảo đảm tính đúng và tính đầy đủ.
CEGAR cut-set	Cut-set mạnh hơn blocking một cycle cụ thể; 2-loop, symmetry và 2-opt.	Dùng làm phase khởi đầu nhanh và làm ngôn ngữ chung cho cut pool.
CCaDical variants	Kết quả lịch sử báo cáo coverage rất cao.	Giữ làm mốc công bố; chưa có executable replay hợp lệ trong repository sau khi wrapper cardinality bị phát hiện sai giao thức.
Sinz variants	Có các unique win so với CCaDical và dùng CNF chuẩn.	Là CEGAR portable đang chạy; mọi so sánh với CCaDical phải ghi rõ đây không phải cùng encoding.
HCup outside-mergecutset	Áp dụng 2-opt exhaustively trước khi sinh cut-set và giữ inprocessing của CaDiCaL.	Là baseline CEGAR mạnh cần chạy cùng điều kiện trước khi chọn làm Phase A.
HCup HFP	Prevent-subcycle can thiệp trên assignment chưa hoàn chỉnh; cấu hình đơn giải 917 graph.	Candidate cho Phase C; phải đo unique win trên 44 timeout của cấu hình CEGAR-957.
HCup inside-mergecutset	Kiểm tra model hoàn chỉnh bên trong một lần gọi SAT và giữ assignment trail; giải 871 graph.	Phân biệt với HFP: "inside" không có nghĩa là phát hiện subcycle trên assignment chưa hoàn chỉnh.
HCup user decision	`hfp-heat` giải 842 graph, thấp hơn HFP prevent-subcycle.	Chỉ nên là ablation phụ, không phải module mặc định.
HCup pair-cutset	Hai biến thể công bố chỉ giải 399 và 754 graph; clause có thể rất lớn và dễ thỏa.	Xem như kết quả âm và không đưa vào pipeline mặc định.
ASP	Trong portfolio cố định, ASP bổ sung 8 graph sau cấu hình CEGAR-957.	Thử component hoặc reachability cuts trong SAT; chưa được gán tám thắng lợi này cho một cơ chế cụ thể nếu chưa có trace.
Picat	Có một unique win đã ghi nhận trên graph725.	Chưa đủ bằng chứng cơ chế; cần source và trace trước khi kết luận kỹ thuật nào tạo ra lợi ích.

Nguồn	Kỹ thuật đáng học	Cách đưa vào phương pháp chung
HCP hiện tại	Nhiều prepass, patcher và decomposition.	Giữ dưới dạng module có gate; không bật toàn bộ mặc định.

6. Phương pháp đề xuất: Adaptive Hybrid SAT-HCP

6.1. Một phương pháp, không phải một túi solver

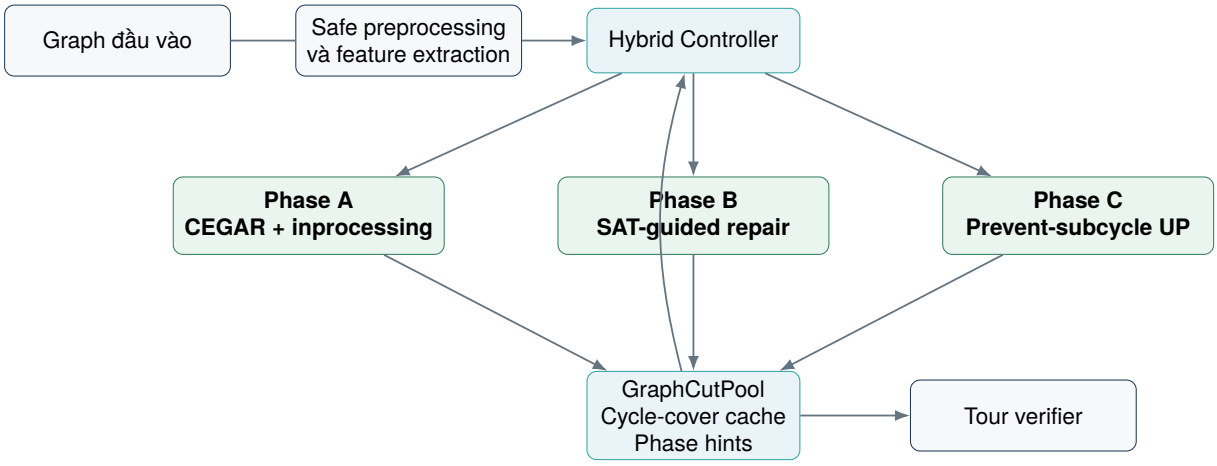
Phương pháp đề xuất có một executable và một controller cố định. Bên trong, nó có thể chuyển phase hoặc rebuild SAT state, nhưng mọi quyết định phải dựa trên đặc trưng graph và telemetry quan sát được trong lúc chạy, không dựa trên đáp án của test set.

Ta gọi phương pháp là **Adaptive Hybrid SAT-HCP**, viết tắt **AHS-HCP**.

6.2. Năm nguyên tắc thiết kế

1. **SAT là lõi chính xác:** mọi heuristic chỉ đề xuất hoặc sửa lời giải; tour cuối phải được kiểm chứng.
2. **Cut ở mức đồ thị là tri thức chung:** cut-set được lưu bằng đỉnh/cạnh, có thể mã hóa lại khi đổi encoding.
3. **Thích nghi có kiểm soát:** chỉ bật module khi tín hiệu cho thấy nó phù hợp.
4. **Tổng ngân sách là cố định:** mọi phase chia sẻ cùng giới hạn 1800 giây.
5. **Có thể giải thích:** log phải nói rõ vì sao controller chuyển phase.

6.3. Kiến trúc tổng thể



Hình 6.1.: Kiến trúc đích: các backend trao đổi tri thức qua cut pool và được điều phối bởi một controller duy nhất.

Trạng thái cài đặt

AHS v1 đã có driver cố định để chọn lộ trình từ đặc trưng graph, chia một ngân sách tổng cho ba backend portable, dừng tiến trình con khi hết giờ, kiểm chứng tour độc lập và lưu manifest/raw log/summary. Ba backend mặc định là CEGAR–Sinz trong `cegar-fix`, HCup `outside-mergecutset` và HCup `hfp-preventsubcycle`. Một treatment thứ tư, `cegar_sinz_population_guided`, đã chạy được để ablate recombination giữa các cycle cover SAT. Treatment này chưa nằm trong `rule_v2`. Tích hợp giữa các backend vẫn ở mức tiến trình; chúng chưa trao đổi cut hoặc SAT state. `cegar-fix` có prototype sub-HCP repair và CRE có prototype `--cegar`, nhưng GraphCutPool, Phase B và CRE–CEGAR chưa được nối vào policy mặc định. Vì vậy sơ đồ trên là kiến trúc đích; phần đang chạy gồm controller tuần tự, verifier chung và population nội bộ tùy chọn trong CEGAR.

Không có học máy trong source hiện tại

Controller AHS v1 là một bộ luật tất định. Nó chỉ dùng m/n và tỷ lệ đỉnh bậc 2 để chọn route; không train, không nạp model và không cần thư viện machine learning. Tệp policy ghi rõ `type=fixed_rules` và `uses_machine_learning=false`; driver từ chối config vi phạm hai điều này.

Sửa sai backend replay

Lần chạy 33 graph trước dùng `cegar-ffi` với một wrapper chuyển luồng lệnh cardinality riêng sang API thêm clause CNF thông thường. Hai giao thức không tương đương, nên ràng buộc exactly-one của lần chạy đó không hợp lệ. AHS v1 loại backend này khỏi policy, dùng Sinz CNF chuẩn và chạy smoke test có verifier trước benchmark dài. Kết quả `UNKNOWN` cũ chỉ dùng để chẩn đoán, không dùng để so sánh với kết quả công bố của CCaDical.

Policy tổng quát và policy tập dư

`rule_v2` là policy tổng quát cho instance chưa biết và có cả ba backend. `residual_v1` chỉ dùng trên 33 graph nằm ngoài union lịch sử 968; nó dành ngân sách cho hai HCup mode để tìm coverage bổ sung. Việc một graph thuộc core-33 không tự chứng minh binary CEGAR hiện tại sẽ timeout trên máy mới. Vì vậy đánh giá phương pháp vẫn phải chạy CEGAR-only, HCup-outside-only và HFP-only dưới cùng điều kiện. Tỷ lệ và thứ tự hiện tại là giả thuyết cần ablation, chưa phải giá trị tối ưu đã chứng minh.

6.4. Phase A: CEGAR có inprocessing

Phase A hiện có hai CEGAR portable. Lựa chọn thứ nhất là `cegar-fix` với Sinz CNF. Lựa chọn thứ hai là HCup chạy stage `outside` và blocking `merge-cutset`. CaDiCaL được phép dùng inprocessing để rút gọn công thức và cấu trúc tìm kiếm.

Mỗi cycle cover được xử lý theo thứ tự:

1. kiểm tra có phải một chu trình Hamilton hay không;
2. thử 2-opt hoặc restricted 3-opt khi xác suất merge cao;
3. sinh cut-set cho các component còn lại;
4. ghi telemetry rồi gọi SAT tiếp.

HCup `outside-mergecutset` áp dụng 2-opt exhaustively rồi mới sinh cut-set cho các component còn lại. `cegar-fix` dùng tùy chọn 2-opt riêng. Hai cách này phải được báo cáo thành hai cấu hình khác nhau cho đến khi một ablation trực tiếp cho thấy có thể thay thế lẫn nhau.

6.5. Treatment đã triển khai: population-guided CEGAR

Memetic algorithm thường giữ nhiều lời giải ứng viên thay vì chỉ một lời giải. Nó ghép hai ứng viên để giữ lại những cạnh hữu ích của cả hai, sau đó dùng local search để sửa. Ý tưởng này được đưa vào CEGAR theo cách hẹp và có thể kiểm chứng, lấy cảm hứng từ Ali và Moscato [3].

Ở đây mỗi model SAT là một *cycle cover*: mọi đỉnh đã có đúng một cạnh vào và một cạnh ra, nhưng graph có thể còn nhiều vòng rời nhau. Module `population_patcher.rs` giữ tối đa 13 cover có tập cạnh khác nhau. Khi có cover mới, nó làm bốn việc:

1. chọn tối đa bốn cover cũ có nhiều cạnh nối giữa các vòng hiện tại và đủ khác cover mới;
2. thử 2-opt với hai cạnh nối lấy từ cover cha;
3. nếu còn ít nhất ba vòng, thử chained k-opt trên hợp cạnh của cover hiện tại và cover cha;
4. chỉ nhận offspring khi số vòng giảm, mỗi đỉnh xuất hiện đúng một lần và mọi cạnh đều thuộc graph.

Để một model SAT khó không giữ toàn bộ thời gian của CEGAR, recombination dùng deadline chung của solver. Mỗi parent có tối đa 32 vòng luân phiên; guided matching và chained LK lần lượt có tối đa 64 và 32 vòng trong mỗi lần gọi. Cả hai local search đều kiểm tra deadline trong lúc duyệt. Nếu hết giờ mà chưa có tour, backend trả `UNKNOWN`; nếu chỉ chạm giới hạn vòng, CEGAR tiếp tục với cover tốt nhất đã qua kiểm tra. Các giới hạn này là hàng rào thực thi, chưa phải tham số đã được chứng minh tối ưu.

Nếu nhiều lần thử liên tiếp không giảm được số vòng, patcher giãn các lần thử theo dãy 1, 3, 7, 15, 31 và tối đa 63 model. Trong thời gian chờ, cover mới vẫn được lưu; một lần cải thiện đặt khoảng chờ về 0. Cơ chế này tránh lặp local search không hiệu quả ở mỗi model SAT.

Controller đặt timeout nội bộ sớm hơn hard kill của tiến trình con một khoảng tối đa một giây. Khoảng nhỏ này cho phép backend in trạng thái và telemetry khi hết ngân sách; cùng quy tắc được áp dụng cho baseline và treatment.

Ta có thể hình dung SAT đã tìm hai bản nháp khác nhau. Mỗi bản nháp đều chưa phải một vòng lớn, nhưng một cạnh trong bản thứ nhất có thể nối hai vòng đang tách rời trong bản thứ hai. Recombination thử đúng các cạnh bổ sung đó trước, thay vì quét toàn bộ graph.

Ranh giới tính đúng

Population patcher không được trả lời `UNSAT`. Nếu ghép thất bại, kết quả chỉ là không cải thiện và CEGAR tiếp tục. Nếu ghép thành một tour, tour vẫn phải qua verifier trên graph gốc. Với graph đã contraction, patcher không được cắt cạnh đại diện cho chuỗi degree-2 cưỡng bức.

Cut sinh từ offspring vẫn sound. Lý do là mỗi vòng con tạo ra một tập đỉnh S khác rỗng và khác toàn bộ V. Mọi chu trình Hamilton phải có ít nhất một cạnh đi từ S ra $V \setminus S$ và ít nhất một cạnh đi ngược lại. Mệnh đề này đúng cho mọi tập S, không yêu cầu offspring phải là model SAT hiện tại.

Baseline là `cegar_sinz_2loop_2opt3`. Nó đặt `--population-patcher 0`; treatment chỉ đổi thành 1. Nhờ vậy A/B đo đúng phần đóng góp của population. Các số smoke test cũ được tạo trước khi có deadline, hard cap, thứ tự cut xác định và backoff nên chỉ chứng minh prototype từng kích hoạt; chúng không dùng để đánh giá phiên bản hiện tại. Chưa được kết luận treatment nhanh hơn trước khi chạy core-33 và easy-control cùng điều kiện.

6.6. Phase B: SAT-guided repair

Prototype gần nhất nằm trong tệp `parallel_sub_hcp.rs`. Khi CEGAR stall, nó gom nhiều subcycle thành cluster, dựng induced subgraph và chạy một CaDiCaL mới với mini-CEGAR để tìm cycle thay thế. Nó chưa phải Phase B đã được chấp nhận của AHS vì chưa có ablation và còn hai vấn đề thực nghiệm: có thể tạo nhiều thread, và SAT call con chưa có hard terminator theo ngân sách cluster.

Thiết kế đích của Phase B là một bài toán SAT nhỏ hơn:

- giữ các cạnh "tốt" bằng assumptions;
- chỉ mở các cạnh quanh biên giữa các subcycle;
- nếu bài toán nhỏ UNSAT, dùng thông tin thất bại để mở rộng vùng sửa;
- nếu tìm được tour, kiểm chứng trực tiếp trên graph gốc.

Đây là một dạng large-neighborhood search nhưng lõi tìm kiếm vẫn là SAT. Trước khi đưa vào controller, prototype phải bị giới hạn một core, có hard timeout và được ablate riêng trên bốn graph B2a cùng một tập đối chứng có cấu trúc tương tự.

6.7. Phase C: prevent-subcycle user propagation

User propagator theo dõi assignment chưa hoàn chỉnh. Nếu một cạnh sắp đóng một chu trình nhỏ trước khi chứa tất cả đỉnh, propagator có thể đẩy literal tương ứng về false và cung cấp lý do cho solver. Trong cấu hình HCup `hfp-preventsubcycle`, propagation này diễn ra trong quá trình CDCL, còn model hoàn chỉnh vẫn được refine ở stage `outside` bằng 2-opt và merge-cutset.

Trực giác

CEGAR thường đợi SAT solver dựng xong một cycle cover sai rồi mới sửa. User propagation giống như người giám sát phát hiện vòng nhỏ đang sắp khép lại và ngăn nó ngay trong lúc xây dựng.

Con số 917/1001 của HFP chưa cho biết nó giải graph nào mà cấu hình CEGAR-957 bỏ sót. Phase C chỉ được coi là có tính bổ sung sau khi có log per-instance cùng điều kiện chạy.

6.8. Ứng viên encoding rebuild: CRE–CEGAR

Chinese Remainder Encoding (CRE) gắn cho mỗi đỉnh một "đồng hồ vị trí" theo nhiều mô-đun nhỏ. Nếu cạnh $u \to v$ được chọn, đồng hồ của v phải tiến một bước so với u. Với tích mô-đun M, một subcycle không chứa đỉnh xuất phát chỉ có thể có độ dài chia hết cho M. Vì vậy CRE có thể loại nhiều vòng nhỏ ngay trong SAT, trước khi CEGAR nhìn thấy một model hoàn chỉnh.

Trực giác về CRE

CEGAR dựng một cycle cover rồi mới cắt các vòng sai. Prevent-subcycle UP cố ngăn vòng sai khi nó sắp khép. CRE đặt thêm các đồng hồ modulo lên đỉnh, khiến nhiều vòng sai tự mâu thuẫn với phép "cộng một" của đồng hồ.

CRE được triển khai thử như một *SAT mode có CEGAR*, không phải một solver trả lời ngay sau model đầu tiên:

1. giữ biến cạnh và degree constraints;

2. thêm các ràng buộc vị trí CRE với một M đã chốt;
3. kiểm tra mọi model bằng tour verifier;
4. nếu model có nhiều cycle, sinh cut-set cho các component rồi gọi SAT tiếp;
5. khi rebuild với M khác, import lại graph cuts đã học.

Nếu $M < |V|$, CRE vẫn có thể trả về nhiều cycle. Model đó chỉ là một ứng viên để refine, không phải lời giải HCP. Chế độ `--cegar` hiện phân rã toàn bộ cycle cover, thêm cut cung vào/cung ra cho từng component rồi giải tiếp; chế độ one-shot vẫn được giữ để tái lập pilot cũ. Log pilot báo `VERIFIED` trên 18/18 graph với $M = 420$, nhưng 17 graph trong số đó vẫn có $M < |V|$ và cả 18 graph đã được CEGAR-957 giải. Kết quả này không chứng minh một ngưỡng tổng quát hoặc marginal gain. Do đó CRE–CEGAR vẫn chưa được đưa vào policy mặc định.

6.9. Vì sao cần rebuild giữa Phase A và Phase C?

Inprocessing có thể đổi, loại hoặc gộp biến nội bộ. User propagator lại cần ánh xạ ổn định giữa biến SAT và cạnh graph. Do đó hai lợi ích này không luôn tương thích trong cùng một state.

Giải pháp đề xuất:

1. kết thúc Phase A tại một điểm chuyển an toàn;
2. xuất mọi graph cut độc lập encoding;
3. tạo SAT state mới ở chế độ tương thích IPASIR-UP;
4. mã hóa lại degree constraints và import graph cuts;
5. dùng cycle cover tốt nhất làm phase hint.

Learned clause nội bộ không phải lúc nào chuyển được, nhưng cut-set ở mức graph thì chuyển được. CaDiCaL tự quản lý việc giảm learned-clause database; API ứng dụng không cho phép chọn và xóa tùy ý learned clause nội bộ. Export/import learned clause ngắn chỉ nên thử khi hai state dùng cùng ánh xạ biến, và phải là một ablation riêng.

7. Hybrid Controller quyết định như thế nào?

7.1. Telemetry tối thiểu

Sau mỗi vòng CEGAR, controller ghi:

- số subcycle và phân bố kích thước;
- số subcycle được 2-opt/3-opt hợp nhất;
- số vòng liên tiếp không giảm cycle count;
- thời gian SAT call gần nhất và tỷ lệ tăng;
- số cut-set, độ dài cut và số blocking clause;
- thời gian dành cho từng patcher;
- với population: số cover đang giữ, số parent đã thử, số lần cải thiện, tổng số subcycle đã giảm, số vòng local search, số lần chạm giới hạn vòng và số lần hết deadline hoặc bỏ qua do backoff;
- SAT mode hiện tại; với CRE ghi thêm M, số biến/clause vị trí và số model nhiều cycle;
- graph density m/n, tỷ lệ đỉnh bậc 2, max degree và số hub.

7.2. Controller đang chạy: luật cố định, không học máy

Phiên bản đầu nên dùng luật giải thích được:

Tín hiệu	Diễn giải khả dĩ	Hành động
$m/n \geq 3$, nhiều hub	Dense graph; SAT branching và clause size có thể là nút thắt.	Duy trì inprocessing, ưu tiên merge-cutset; chỉ bật decomposition sau khi có stall.
Nhiều đỉnh bậc 2, merge progress bằng 0	Cycle cover dễ rơi vào các vòng nhỏ cưỡng bức.	Chuyển sang prevent-subcycle UP hoặc connectivity cuts mạnh hơn.
Ít subcycle lớn, patcher liên tục giảm cycle count	Lời giải đang gần một tour.	Cho repair thêm ngân sách; chưa rebuild solver.
Nhiều subcycle nhỏ lặp lại	Patcher có thể quay lại cùng attractor.	Giảm patching, thêm cut sớm và đa dạng phase.
SAT call cuối chiếm phần lớn runtime	State có dấu hiệu hardening.	Export cut pool rồi rebuild với UP hoặc một encoding khác như CRE–CEGAR.

Bảng 7.1.: Các luật controller cần ablate. Bản v0 hiện chỉ dùng hai ngưỡng cấu trúc đã khóa trong policy.

Các lớp A/B1/B2a/B2b mô tả động lực của một lần chạy, không phải nhãn bất biến của graph. Bảng sau chỉ là bản đồ giả thuyết để chọn thí nghiệm:

Lớp	Trạng thái	Cơ chế nên thử	Mức bằng chứng
A	Đã giải 8/8	Giữ CEGAR hiện tại và kiểm tra regression.	Cao.
B1	Còn 6 core	Inprocessing, merge-cutset, encoding và decomposition.	Trung bình; UP có thể mất lợi ích inprocessing nên phải đo.
B2a	Còn 3 core	SAT-guided repair, prevent-subcycle, path-cover hoặc decomposition.	Thấp; chưa có run trực tiếp.
B2b	Còn 24 core	Prevent-subcycle, CRE–CEGAR, rebuild có cut transfer và đổi encoding/variable order.	Thấp đến trung bình; mới có telemetry, chưa có run CRE trên core.

Bảng 7.2.: Mapping giả thuyết từ failure class sang thí nghiệm.

Không đặt ngưỡng từ trực giác rồi báo trên cùng tập dữ liệu. Mỗi ngưỡng phải được chọn trên training split theo họ cấu trúc, khóa lại và sau đó mới chạy test split.

7.3. Phân bổ ngân sách

Không nên cho mỗi backend 1800 giây rồi cộng kết quả, vì như vậy phương pháp hybrid được hưởng nhiều tài nguyên hơn baseline. Một cách công bằng là:

$$T_{\text{preprocess}} + T_A + T_B + T_C + T_{\text{CRE}} \leq 1800 \text{ giây}.$$

Controller có thể chuyển phase theo sự kiện thay vì một mốc thời gian cứng. Ví dụ, Phase A tiếp tục khi vẫn tạo tiến bộ; nó chuyển sớm nếu nhiều vòng liên tiếp không cải thiện và SAT call tăng nhanh.

Không dùng oracle trá hình

Không được nhìn tên graph rồi tra bảng "solver nào từng thắng". Quy tắc phải dựa trên đặc trưng tính được từ graph hoặc telemetry sinh ra trong lần chạy hiện tại. Nếu không, kết quả chỉ là memorization của benchmark.

8. Các hướng brainstorming có tiềm năng

8.1. Encoding lai theo từng đỉnh

Không nhất thiết mọi đỉnh phải dùng cùng một cardinality encoding:

- bậc thấp: pairwise hoặc binomial, ít biến phụ;
- bậc trung bình: Sinz, lan truyền ổn định;
- hub bậc cao: commander hoặc encoding phân cấp.

Đây là một công thức SAT duy nhất nhưng được thiết kế theo cấu trúc cục bộ của graph.

8.2. Bốn đề xuất từ nhánh SAT encoding

Tài liệu NHCP nêu bốn cách tăng cường CRE. Không nên bật cả bốn cùng lúc. Trạng thái phù hợp của từng cách là:

Đề xuất	Quyết định	Lý do
Symmetry breaking tại đỉnh xuất phát	Giữ và ablate	Đã có trong CRE gốc, nên không tính là cải tiến mới. Chỉ an toàn cho graph vô hướng hoặc digraph có đối xứng đảo chiều.
Chọn đỉnh xuất phát bậc nhỏ nhất	Giữ như một policy	Phá đối xứng quay và có thể tăng propagation, nhưng lợi ích của tiêu chí bậc nhỏ nhất chưa được đo riêng.
Preprocessing theo khoảng cách	Sửa trước khi dùng	Khoảng tuyệt đối chỉ an toàn khi vị trí được mã hóa duy nhất. Với CRE modulo, phải lọc residue theo tập khả dĩ.
PBLib cho ExactlyOne	Ablation, không mặc định	PBLib chỉ là bộ sinh encoding. Cần so clause, biến phụ, propagation, runtime và chi phí dependency với pairwise/Sinz/commander hiện có.

Bảng 8.1.: Cách tiếp nhận bốn đề xuất từ nhánh SAT encoding.

8.2.1. Preprocessing khoảng cách an toàn cho CRE

Gọi $d_s(v)$ là khoảng cách ngắn nhất từ đỉnh xuất phát s đến v, và $d_v(s)$ là khoảng cách từ v về s. Vị trí thật của v trong một tour phải thuộc

$$L_v = d_s(v) + 1, \qquad U_v = |V| - d_v(s) + 1.$$

Với một phân đoạn modulo q, chỉ được cấm residue không xuất hiện trong khoảng này:

$$R_v^{(q)} = \{t \bmod q \mid L_v \le t \le U_v\}.$$

Ta thêm clause cấm $\mathrm{pos}_q(v) = r$ chỉ khi $r \notin R_v^{(q)}$. Cách này có thể yếu hơn cắt khoảng tuyệt đối nhưng giữ tính đúng. Ngược lại, nếu $M < |V|$, cấm trực tiếp một giá trị t nằm ngoài khoảng có thể sai vì t và $t + M$ có cùng residue. Ví dụ, với $M = 3$, cấm vị trí 1 cũng vô tình cấm vị trí 4.

Với full binary position, cắt khoảng trực tiếp chỉ an toàn khi miền vị trí không lặp và $M \geq |V|$. Khi đó có thể dùng ràng buộc khoảng đơn điệu của BinaryAdder. Với graph có hướng, symmetry đảo chiều phải tắt và hai khoảng cách phải được tính đúng chiều.

8.2.2. ExactlyOne: dùng policy thay vì khóa vào PBLib

AtLeastOne chỉ cần một clause chứa các cạnh thật sự kề đỉnh. Phần AtMostOne nên được chọn theo bậc: pairwise cho bậc nhỏ, rồi so Sinz, commander, cardinality network hoặc PBLib trên bậc lớn. Prototype có `EOPbLib.exactly_one()`, nhưng `at_most_one()` vẫn là stub; CRE độc lập vẫn dùng AtMostOne tự viết. Chưa có thí nghiệm chứng minh PBLib thắng trên full 1001.

8.3. GraphCutPool độc lập encoding

Mỗi cut được lưu ở dạng cấu trúc:

$$(S,\ \delta(S),\ \text{nguồn},\ \text{số lần hữu ích},\ \text{thời điểm tạo}).$$

Khi rebuild, cut được dịch sang biến của encoding mới. Có thể xếp hạng cut theo độ ngắn, số lần propagation hoặc mức độ tái xuất hiện của component.

8.4. Phase hints từ cycle cover tốt nhất

Khi tạo SAT state mới:

- ưu tiên true cho cạnh thuộc cycle cover tốt nhất;
- ưu tiên false cho cạnh nhiều lần đóng subcycle xấu;
- giảm dần độ tin cậy sau mỗi lần thất bại để tránh khóa vào attractor.

Phase hint chỉ đổi giá trị solver ưu tiên thử trước; nó không bắt buộc cạnh phải được chọn. Ngược lại, assumption là ràng buộc tạm thời nhưng bắt buộc trong một lần gọi SAT. Không nên đưa toàn bộ cycle cover vào assumptions vì điều đó gần như khóa solver vào lời giải cũ. Nếu thử assumptions trên một tập cạnh nhỏ, khi UNSAT phải bỏ hoặc nới tập này rồi tiếp tục.

8.5. Tổng quát hóa cut theo xung đột

Nếu nhiều cycle cover khác nhau liên tục tạo cùng một phân hoạch component, ta có thể tổng quát hóa các blocking clause riêng lẻ thành một connectivity cut mạnh hơn. ASP bổ sung 8 graph trong thứ tự portfolio hiện tại, nhưng log chưa chứng minh kỹ thuật nào tạo ra các thắng lợi đó. Vì vậy connectivity cut là một giả thuyết SAT cần ablation, không phải cơ chế đã xác nhận từ ASP.

8.6. Hướng tương lai chưa triển khai: học controller bằng dữ liệu

Phần này không thuộc source chạy hiện tại. Chỉ sau khi có log đầy đủ, một mô hình nhỏ mới có thể được thử để dự đoán:

- encoding khởi đầu;

- có nên bật user propagation;
- thời điểm chuyển phase;
- patcher nào đáng chạy;
- cách chia ngân sách.

Tuy nhiên, 1001 instance là tập dữ liệu nhỏ và chứa nhiều graph cùng họ. Các lớp hiếm còn nhỏ hơn: Class A có 8 mẫu và B2a có 4 mẫu trong baseline timeout. Một increment đầu tiên cũng chưa đủ để nhận ra các động lực được định nghĩa qua nhiều increment. Phải chia train/test theo *họ cấu trúc*, không chia ngẫu nhiên từng graph. Nếu graph950 được train còn graph963 gần như cùng cấu trúc nằm trong test, kết quả có thể bị leakage.

9. Thiết kế thực nghiệm công bằng

9.1. Câu hỏi nghiên cứu

RQ1. Một controller đơn có vượt cấu hình SAT-CEGAR đơn tốt nhất 957/1001 không?

RQ2. Cut transfer giữa phase có tạo unique wins hay chỉ thêm overhead?

RQ3. Prevent-subcycle UP có bổ sung cho CEGAR inprocessing trên core-33 không?

RQ4. SAT-guided repair có thắng 2-opt/3-opt thuần trên nhóm merge stall không?

RQ5. Encoding lai theo đỉnh có giảm runtime mà không gây regression trên easy set không?

RQ6. Trên B2b, rebuild giữ graph cuts có giảm hardening đủ để bù chi phí mất learned clauses không?

RQ7. CRE–CEGAR có tạo unique win so với CEGAR-957, và cải tiến nào trong symmetry, start policy, distance filtering hoặc AMO policy tạo ra lợi ích đó?

9.2. Ba mức kết quả phải báo cáo riêng

1. **Single static SAT:** cấu hình cố định, mốc tham chiếu 957.
2. **Adaptive single-method SAT:** AHS-HCP, một executable và một ngân sách tổng.
3. **Unrestricted portfolio:** có thể bao gồm ASP/Picat, dùng để đo coverage thực dụng tối đa.

9.3. Tập đánh giá trọng tâm

Ngoài full set 1001 graph, cần theo dõi:

- **Core-33:** chưa được 21 phương pháp có log per-instance đầy đủ giải; trạng thái trong HCup hiện chưa biết.
- **SAT-only open-38:** core-33 cộng graph677, graph725, graph744, graph810 và graph940, vốn đang được ASP/Picat bổ sung.
- **CRE pilot:** 18 graph legacy chỉ dùng để kiểm tra tương thích và tái lập; kết luận coverage phải dựa trên core-33, 44 miss của CEGAR-957 và full 1001.
- **Easy-overhead set:** phải được định nghĩa và khóa trước benchmark. Tập `10-300-10` hiện chỉ là pilot 30 graph, không đại diện cho toàn bộ graph dễ.
- **Các họ cấu trúc:** dense B1, merge-stall B2a, long-call B2b và attractor A. Đây là failure dynamics của run baseline; không mặc định là nhãn bất biến của graph.

9.4. Chỉ số đánh giá

Chỉ số	Ý nghĩa
Solved count	Số instance có tour hợp lệ trong cap. Đây là chỉ số chính.
PAR-2	Runtime nếu giải được; phạt 2× timeout nếu không giải. Phản ánh cả tốc độ và độ bền.
Median/p90	Tốc độ trên phần instance đã giải. Không thay thế solved count.
Unique wins	Instance chỉ phương pháp này giải được; cho thấy tính bổ sung thực.
Marginal gain	Số instance một phase bổ sung sau các phase trước.
Phase overhead	Thời gian controller, rebuild, encoding, patching và verification.

Solved count và PAR-2 là hai chỉ số chính. Với kết quả paired trên cùng instance, chênh lệch solved/timeout có thể kiểm tra bằng exact McNemar. PAR-2 nên kèm paired bootstrap hoặc permutation interval. Paired Wilcoxon chỉ dùng như phân tích phụ trên các graph mà cả hai cấu hình đều giải; nếu chỉ nhìn runtime của phần solved, kết quả sẽ thiên lệch vì bỏ qua timeout. Có thể bổ sung survival analysis để xử lý runtime bị censor tại cap.

Easy-overhead phải báo median, p90 và số graph chậm hơn các ngưỡng 5%, 10% và 25%. Không dùng một con số như “944 easy graph” nếu chưa có tiêu chí tạo tập và danh sách cố định.

9.5. Ablation bắt buộc

Cấu hình	Cut pool	Repair SAT	UP	Controller
CEGAR base	×	×	×	×
+ cut transfer	✓	×	×	×
+ SAT-guided repair	✓	✓	×	×
+ user propagation	✓	×	✓	×
+ repair + user propagation	✓	✓	✓	×
AHS-HCP đầy đủ	✓	✓	✓	✓

Bảng 9.1.: Ablation giúp xác định module nào thực sự tạo lợi ích.

CRE cần một ablation riêng, luôn giữ cùng solver, seed, cap và M khi so các dòng:

Cấu hình	Thành phần được bật
CRE–CEGAR hiện có	Start vertex bậc nhỏ nhất, symmetry đảo chiều, không distance filtering, AMO hiện tại.
CRE-fixed-start	Thay start bậc nhỏ nhất bằng một start cố định để đo riêng start policy.
CRE-no-sym	Tắt symmetry đảo chiều; đây là control bắt buộc khi đánh giá symmetry.
CRE-distance	Bản hiện có + residue filtering theo khoảng cách an toàn.
CRE-AMO	Bản hiện có + policy chọn AMO theo bậc hoặc PBLib.
CRE-full	Symmetry an toàn + distance filtering + AMO policy; start policy được chọn từ ablation trước.

Bảng 9.2.: Ablation riêng cho nhánh CRE–CEGAR.

Hiện chỉ hàng đầu đã có backend chạy được. Các hàng còn lại là thí nghiệm phải triển khai và kiểm thử riêng; không được gộp chúng thành một claim đã có.

Ảnh hưởng của M phải được đo trong một thí nghiệm khác. Có thể thử các mốc pilot $12, 60, 105, 420$ và một $M_{\text{exact}} > |V|$, nhưng lịch chuyển M phải được khóa trước test và dùng chung một ngân sách. Không lấy giá trị tốt nhất sau khi nhìn kết quả rồi gọi đó là một solver đơn.

Tiêu chuẩn để tuyên bố cải tiến

Phương pháp mới phải được chạy như một cấu hình đã chốt, trên cùng máy, cùng giới hạn CPU/RAM, cùng verifier và cùng cách tính timeout. Không so runtime trực tiếp giữa các CSV được đo trên phần cứng khác nhau.

10. Lộ trình triển khai

10.1. Giai đoạn 0: hoàn thiện bằng chứng

1. Trên core-33, chạy hai cấu hình HCup mạnh nhất để tìm coverage mới ngoài union 968. Hai cấu hình là `outside-mergecutset` và `hfp-preventsubcycle`.
2. Chạy `hfp-preventsubcycle` trên 44 timeout của cấu hình CEGAR-957 để đo marginal gain của Phase C.
3. Chạy pilot CRE hiện có trên core-33 và 44 timeout nói trên. Báo riêng `VERIFIED`, `MULTICYCLE`, `UNSAT` và `TIMEOUT`; không tính model nhiều cycle là solved.
4. Import JSONL vào bảng coverage động và kiểm chứng mọi tour.
5. Nếu có tín hiệu dương, chạy full 1001 cùng máy để đo overlap, regression và overhead.
6. Thu telemetry thống nhất cho CEGAR và HCup.

10.2. Giai đoạn 1: GraphCutPool và telemetry

Đây là bước rủi ro thấp nhất và có giá trị độc lập:

- định nghĩa cut ở mức graph;
- gắn provenance và thống kê sử dụng;
- xuất/import cut khi rebuild;
- bảo đảm mọi cut đều sound bằng unit test trên graph nhỏ.

10.3. Giai đoạn 1a: ablate population-guided CEGAR

Phần code của treatment đã hoàn tất; phần bằng chứng chưa hoàn tất. Hai cấu hình A/B là:

- baseline: `ahs_cegar_only`;
- treatment: `ahs_population_cegar_only`.

Hai cấu hình phải chạy trên cùng máy, cùng cap và cùng thứ tự instance. Trước tiên chạy core-33 để tìm marginal coverage, sau đó chạy `easy_overhead` để đo regression. Chỉ đưa treatment vào policy tổng quát nếu nó có thêm nghiệm hoặc cải thiện PAR-2 mà không gây thoái lui lớn trên tập đối chứng.

10.4. Giai đoạn 2: kiểm chứng và ablate CRE–CEGAR

Prototype CRE đã có nên có thể kiểm tra trước khi port sâu:

1. kiểm tra vòng verify–cut–solve hiện có bằng brute force trên graph nhỏ, gồm cả trường hợp cần nhiều increment;
2. chạy qua benchmark runner để external timeout bao phủ cả thời gian sinh encoding;
3. chỉ dùng preprocessing khoảng cách theo residue; tắt symmetry đảo chiều trên graph có hướng;
4. kiểm tra exhaustive trên graph nhỏ bằng cách so với brute force, cả khi từng cải tiến bật và tắt;
5. chạy bảng ablation CRE trên pilot, core-33, 44 miss và một easy-control set;
6. chỉ đưa CRE vào controller nếu có unique win hoặc giảm PAR-2 ổn định sau khi tính cả thời gian sinh CNF và rebuild.

10.5. Giai đoạn 3: làm cứng và ablate Phase B

Làm cứng prototype SAT-guided repair trước khi nối vào pipeline:

1. chạy tuần tự hoặc giới hạn rõ một CPU core;
2. gắn terminator theo deadline cho mọi CaDiCaL con;
3. chạy trên bốn B2a: graph479, graph809, graph868 và graph960;
4. tạo tập đối chứng có mật độ cạnh và tỷ lệ đỉnh bậc 2 tương tự;
5. so với 2-opt/3-opt thuần dưới cùng ngân sách;
6. chỉ giữ module nếu nó tạo marginal gain hoặc giảm PAR-2 ổn định.

10.6. Giai đoạn 4: driver A–C và tích hợp HCup

Có hai mức tích hợp:

Mức thực nghiệm nhanh. AHS v1 đã có driver cố định điều phối ba backend trong một ngân sách tổng và kiểm chứng tour. Bước kế tiếp của mức này là bổ sung file cut/cycle-cover chung sau khi các CLI có giao diện export/import tương thích.

Mức phương pháp hoàn chỉnh. Port prevent-subcycle hoặc đóng gói HCup thành library có API ổn định. Khi đó controller, cut pool và verifier nằm trong một executable.

Nên chứng minh tính bổ sung ở mức driver trước, rồi mới trả chi phí port sâu.

10.7. Giai đoạn 5: mở rộng controller luật; selector học là tùy chọn tương lai

Khi CRE, Phase B và driver A–C đã có số liệu, controller ban đầu chỉ cần các state `FAST_CEGAR`, `CRE_REBUILD`, `SAT_REPAIR`, `UP_REBUILD`, `FINAL_FALLBACK`, `SOLVED` và `TIMEOUT`. Mọi chuyển state phải ghi lý do, thời gian còn lại và snapshot telemetry.

Chỉ sau khi controller luật ổn định mới được cân nhắc model học. Model đó không phải điều kiện để hoàn thành AHS rule-based. Mỗi ý tưởng cần một ablation riêng; nếu gộp nhiều thay đổi cùng lúc, ta không biết nguyên nhân của thắng hoặc thua.

11. Pseudocode dễ đọc

Pseudocode dưới đây mô tả hệ đích sau khi các phase đã được cài đặt và vượt qua ablation. Nó không phải mô tả của binary hiện tại.

Thuật toán AHS-HCP

```
INPUT: graph G, total budget T
OUTPUT: verified Hamiltonian cycle, or UNKNOWN

1. preprocess G using only sound reductions
2. compute structural features
3. create GraphCutPool and HybridController
4. start Phase A: CEGAR with inprocessing
5. while remaining time > 0:
   a. ask SAT for a candidate cycle cover
   b. if candidate is one cycle: verify and return it
   c. try the repair selected by Controller
   d. if repair creates one cycle: verify and return it
   e. generate sound graph cuts and store them
   f. update telemetry
   g. if Controller requests a phase change:
      - export graph cuts and best cycle cover
      - rebuild the selected SAT mode (CEGAR, CRE--CEGAR or UP)
      - import cuts and phase hints
6. return UNKNOWN
```

12. Câu hỏi thường gặp

Phương pháp này có còn là SAT-only không?

Có, nếu ASP và Picat chỉ được dùng làm nguồn ý tưởng hoặc baseline, còn mọi phase tìm kiếm của AHS-HCP đều dựa trên SAT/CaDiCaL, cut-set và user propagation.

Rebuild solver có làm mất hết công sức cũ không?

Một phần learned clause nội bộ có thể mất, nhưng graph cuts, cycle cover tốt nhất, phase hints và telemetry vẫn được giữ. Mục tiêu nghiên cứu là chứng minh phần tri thức chuyển được bù lại chi phí rebuild.

Tại sao không chạy CEGAR và HCup song song?

Chạy song song có thể tăng coverage thực dụng, nhưng dùng nhiều CPU hơn baseline. Đây có thể là một chế độ portfolio đa lõi riêng, không nên dùng làm kết quả chính cho phương pháp một lõi.

Có nên dùng học máy ngay không?

Không trong phiên bản này. Source hiện tại chỉ cài controller luật. Trước hết cần log đầy đủ và baseline này; thêm học máy quá sớm sẽ khó phân biệt lợi ích thật với overfitting vào 1001 graph.

Nếu trả về SAT thì đã chắc chắn đúng chưa?

Chưa. Luôn phải kiểm tra rằng tour chứa đúng mọi đỉnh một lần và mọi cặp liên tiếp là cạnh hợp lệ, kể cả cạnh quay về điểm đầu.

A. Bảng thuật ngữ

Thuật ngữ	Giải thích ngắn
Assignment	Một cách gán đúng/sai cho các biến SAT.
Baseline	Phương pháp tham chiếu dùng để so sánh.
Blocking clause	Clause được thêm để ngăn solver lặp lại một cấu hình sai.
CaDiCaL	SAT solver CDCL hiện đại được dùng trong các phương pháp đang xét.
CEGAR	Vòng lặp giải abstraction, tìm counterexample rồi refine.
Clause	Phép hoặc của các literal.
CNF	Phép và của nhiều clause.
Cut-set	Tập cạnh đi qua biên của một tập đỉnh.
Cycle cover	Tập nhiều chu trình rời nhau phủ toàn bộ đỉnh.
Encoding	Cách chuyển ràng buộc bài toán sang clause SAT.
Inprocessing	Các phép rút gọn công thức được SAT solver thực hiện trong lúc giải.
IPASIR-UP	Giao diện cho phép user propagator tương tác với SAT solver.
Literal	Một biến hoặc phủ định của biến.
PAR-2	Chỉ số runtime phạt timeout bằng hai lần cap.
Phase hint	Gợi ý giá trị đúng/sai ưu tiên khi solver bắt đầu tìm kiếm.
Subcycle	Chu trình chỉ chứa một phần đỉnh của graph.
Telemetry	Dữ liệu đo động lực tìm kiếm trong lúc chạy.
User propagator	Module ngoài cung cấp propagation hoặc clause chuyên biệt cho bài toán.
VBS	Virtual Best Solver, oracle chọn kết quả tốt nhất sau khi đã biết mọi run.

B. Tái tạo báo cáo coverage và benchmark

Sau khi clone repository, chuẩn bị đúng các source public đã khóa, môi trường PySAT và binary native bằng:

```
./setup_dependencies.sh
./setup_dependencies.sh --check
```

Script đặt HCup và SAT_HCP/CRE trong thư mục `external/`; không cần tự sắp xếp các repository kề nhau. Nếu chỉ chạy CRE trên máy chưa có Rust, dùng `./setup_dependencies.sh --no-build`.

Biên dịch chính tài liệu này bằng XeLaTeX:

```
mkdir -p output/pdf
latexmk -xelatex -interaction=nonstopmode \
  -outdir=output/pdf docs/adaptive-hybrid-sat-hcp.tex
```

Sinh lại các CSV coverage:

```
python3 tools/build_hybrid_coverage.py
```

Chạy đầy đủ hai cấu hình HCup trên core-33 để đo overlap. Không dùng early stop trong thí nghiệm này:

```
python3 tools/hcp_benchmark.py \
  --config benchmarks/hybrid_variants.json \
  --set historical_open_33 \
  --variants hcup_outside_mergecutset,hcup_hfp_preventsubcycle \
  --output benchmark-runs/hcup-open33
```

Chạy riêng HFP trên 44 timeout của cấu hình CEGAR-957:

```
python3 tools/hcp_benchmark.py \
  --config benchmarks/hybrid_variants.json \
  --set cegar957_timeout_44 \
  --variants hcup_hfp_preventsubcycle \
  --output benchmark-runs/hfp-cegar957-timeout44
```

Chạy CRE–CEGAR $M = 420$ với external timeout và verifier chung:

```
./run_cre_benchmark.sh \
  --set historical_open_33 \
  --timeout 1800 \
  --output benchmark-runs/cre-cegar-m420-open33
```

Chạy A/B cho population-guided CEGAR trên core-33:

```
./run_ahs_population_ablation.sh \
  --timeout 1800 \
  --output benchmark-runs/population-cegar-ablation-open33 \
  --resume
```

Tạo bảng paired theo từng instance và đọc telemetry từ đúng stage bên trong AHS:

```
python3 tools/analyze_population_ablation.py \
  benchmark-runs/population-cegar-ablation-open33/results.jsonl
```

Nếu chỉ cần chạy treatment ở background và quay lại sau:

```
nohup ./run_ahs_population33.sh \
  --timeout 1800 \
  --output benchmark-runs/population-cegar-open33-1800s \
  --resume > population-cegar-open33-1800s.console.log 2>&1 &
```

Import kết quả mới vào coverage động:

```
python3 tools/build_hybrid_coverage.py \
  --benchmark-results benchmark-runs/hcup-open33/results.jsonl
```

C. Tài liệu tham khảo

Tài liệu tham khảo

[1] R. Ohashi, T. Soh, D. Le Berre, H. Nabeshima, M. Banbara, K. Inoue, và N. Tamura. *SAT-Based CEGAR Method for the Hamiltonian Cycle Problem Enhanced by Cut-Set Constraints*. SAT 2025, LIPIcs 341, Article 24, 2025.

[2] F. Funder. *An UPgrade for HCP: Solving the Hamiltonian Cycle Problem via User Propagators*. Diplomarbeit, Technische Universität Wien, 2026.

[3] S. Ali và P. Moscato. *A Memetic Algorithm To Find a Hamiltonian Cycle in a Hamiltonian Graph*. arXiv:2403.07886, 2024.

[4] A. Biere, M. Heule, H. van Maaren, và T. Walsh, biên tập. *Handbook of Satisfiability*. IOS Press, 2009.

[5] R. M. Karp. Reducibility among combinatorial problems. Trong *Complexity of Computer Computations*, 1972.

Tài liệu này đi kèm mã nguồn và pipeline kiểm chứng trong repository HCP-CEGAR.